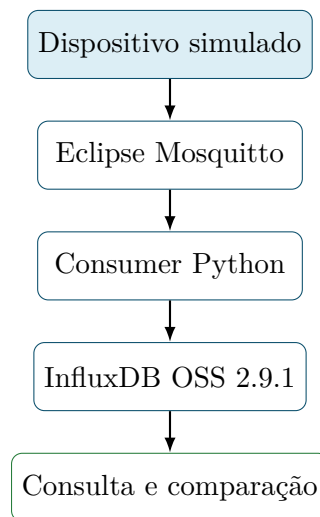

Portos Conectados

Prova de conceito de uma bancada de dados e comunicação

MQTT real, consumer Python e InfluxDB real



Relatório técnico-acadêmico de prova de conceito

8 de outubro de 2026

Resumo

Este relatório apresenta uma prova de conceito, executada integralmente em terminal, para a etapa de software posterior a um futuro Network Server LoRaWAN do projeto Portos Conectados. A bancada implementa um simulador de sondas com continuidade temporal, transporte por MQTT em um broker Eclipse Mosquitto real, consumo e validação em Python, persistência em InfluxDB real e verificação posterior por consulta e comparação dos valores. O critério de aprovação exige que cada mensagem seja publicada, recebida, validada, gravada, localizada por seu identificador e comparada com a leitura original.

No ambiente avaliado, a suíte automatizada obteve 20 testes aprovados e 2 ignorados de forma explícita. O fluxo completo confirmou 100 de 100 mensagens, sem perda, a 20,259 mensagens por segundo; uma execução adicional confirmou 1.000 de 1.000 mensagens, a 17,983 mensagens por segundo. A latência aumentou com a formação de fila: o percentil 95 passou de 2.217,452 ms no teste de 100 mensagens para 28.989,733 ms no teste de 1.000 mensagens. A escrita direta em lote no InfluxDB confirmou 1.000 pontos em todas as configurações testadas e alcançou 22.669,311 pontos por segundo com lote 1.000. Também foram validadas deduplicação, detecção de ordem, fila SQLite durante indisponibilidade do banco, reconexão MQTT e reinício do consumer.

Os resultados caracterizam esta máquina e esta configuração. RSSI, SNR e distância são simulados. Não foi executado ensaio de alcance físico LoRaWAN nem uma topologia em três computadores; portanto, nenhuma conclusão de alcance de rádio é apresentada.

Palavras-chave: Internet das Coisas; qualidade da água; MQTT; InfluxDB; séries temporais; LoRaWAN; teste de software.

Sumário

1	Introdução	1
1.1	Objetivos	1
1.2	Perguntas experimentais	1
2	Escopo e terminologia experimental	1
3	Arquitetura implementada	2
3.1	Componentes	3
4	Contrato e qualidade dos dados	3
4.1	Campos	3
4.2	Tempo, identidade e repetição	4
5	Modelo de dados no InfluxDB	4
6	MQTT e preparação para LoRaWAN	5
7	Metodologia de teste	5
7.1	Ambiente	5
7.2	Critério end-to-end	5
7.3	Métricas	6
7.4	Falhas controladas	6
8	Resultados	6
8.1	Suíte automatizada	6
8.2	Fluxo completo	6
8.3	Escala por número de dispositivos	8
8.4	Escrita em lote no InfluxDB	8
8.5	Frequência solicitada e observada	10
8.6	Falha e recuperação	10
8.7	Evento ambiental simulado	11
9	Discussão	11
9.1	O que foi demonstrado	11
9.2	Ameaças à validade	11
10	Reprodução	11
10.1	Obtenção e preparação	11
10.2	Sequência mínima de validação	12

10.3 Execução cotidiana	13
10.4 Topologia entre computadores	13
11 Substituição futura pelo LoRaWAN real	13
12 Conclusão	14
A Arquivos de evidência selecionados	14
B Referências técnicas	15

1 Introdução

O projeto Portos Conectados pretende monitorar temperatura, pH, turbidez, condutividade e oxigênio dissolvido por sondas que futuramente usarão LoRaWAN. Antes da disponibilidade do rádio, do gateway e do Network Server, é possível investigar experimentalmente a infraestrutura que receberá os uplinks da aplicação. Esta prova de conceito, ou POC, foi construída como uma bancada reproduzível para esse propósito.

A arquitetura futura é composta por sensores, microcontrolador LoRaWAN, enlace de rádio, gateway, Network Server, MQTT, aplicação Python e InfluxDB. A POC substitui somente a parte anterior ao MQTT por um dispositivo simulado. O broker, a comunicação TCP/IP, o consumer, a escrita e a consulta ao banco são operações reais.

O princípio metodológico central é que aceitar uma requisição de escrita não prova a persistência. Uma execução é aprovada somente quando o mesmo `message_id` é encontrado posteriormente no InfluxDB e os valores recuperados são iguais aos valores publicados.

1.1 Objetivos

O objetivo geral é implementar e avaliar uma bancada terminal para estudar o fluxo de dados MQTT–Python–InfluxDB antes da implantação do hardware LoRaWAN. Os objetivos específicos são:

- formalizar o contrato mínimo das mensagens ambientais;
- medir entrega, latência, vazão, perda, duplicidade e ordem;
- comparar diferentes tamanhos de lote de escrita;
- observar o comportamento com mais dispositivos e mensagens;
- manter dados em fila local quando o InfluxDB estiver indisponível;
- verificar reconexão quando o MQTT cair;
- produzir evidências JSON, relatórios TXT/CSV e rastreamento individual;
- preparar a troca do simulador por um Network Server real sem reescrever o núcleo do consumer.

1.2 Perguntas experimentais

Os ensaios foram organizados para responder se o banco recebeu os dados, se eles foram recuperados sem alteração, qual latência e vazão foram observadas, se houve perda ou duplicidade, como o sistema reagiu ao aumento de carga e o que ocorreu nas indisponibilidades. Questões que exigem rádio, gateway ou máquinas fisicamente distintas são mantidas como trabalho futuro.

2 Escopo e terminologia experimental

Três experimentos são conceitualmente separados:

No documento, *medido* identifica uma observação obtida por relógios e contadores do sistema; *calculado* identifica estatísticas derivadas, como média e percentis; *simulado*

Tabela 1: Separação dos experimentos.

Experimento	O que comprova	Situação nesta rodada
Software local	MQTT, processamento, validação, armazenamento, consulta, latência e perdas no ambiente local	Executado
Rede entre computadores	Tráfego TCP/IP real entre hosts distintos	Não executado; houve apenas TCP/IP local
Campo LoRaWAN	Rádio, gateway, PDR, RSSI, SNR e alcance nas condições medidas	Não executado

identifica condições produzidas pelo software; e *não testado* identifica ausência de evidência experimental.

Toda execução do simulador declara:

```
RADIO MODE: SIMULATED
PHYSICAL RANGE VALIDATION: NOT PERFORMED
```

Valores sintéticos de `distance_m`, RSSI, SNR ou perda exercitam o tratamento de metadados. Eles não demonstram alcance de LoRaWAN.

3 Arquitetura implementada

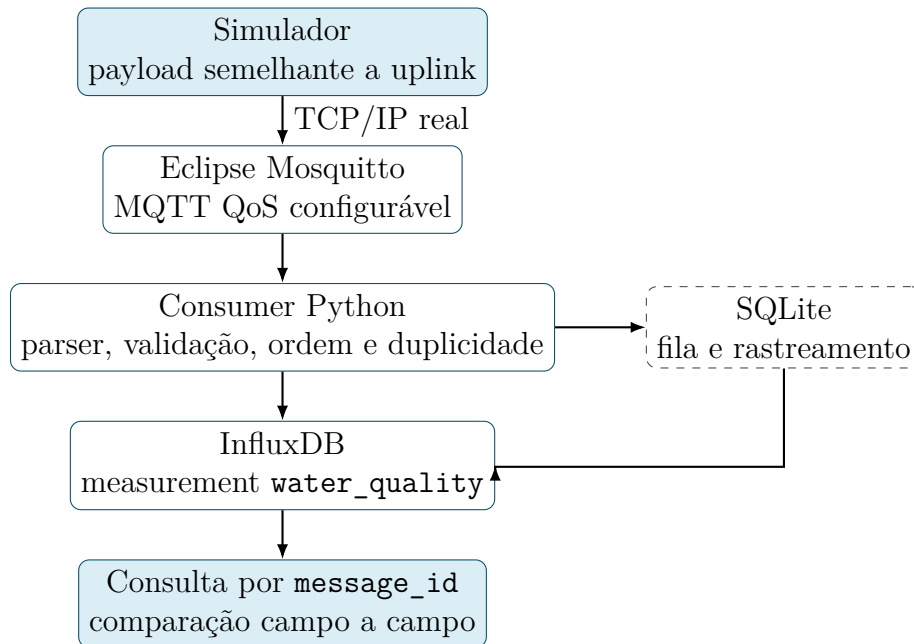


Figura 1: Fluxo implementado na POC.

O payload simulado se aproxima do formato decodificado pelo The Things Stack. O parser está isolado em `consumer/parser.py`; assim, uma integração futura com The

Things Stack ou ChirpStack deve adaptar o envelope externo e preservar o contrato interno.

3.1 Componentes

Simulador

Mantém um estado ambiental por dispositivo, aplica pequenas variações temporais e pode produzir uma alteração forte de turbidez. Cada dispositivo possui identificador e contador próprios.

MQTT

O Mosquitto recebe publicações no tópico configurado. O QoS é configurável e o padrão da bancada é 1.

Consumer

Recebe, converte o envelope, valida formato e valores, detecta repetição e ordem, escreve no banco e registra cada estágio.

Fila local

O SQLite preserva mensagens válidas quando o banco não responde. Uma pendência é removida apenas depois de escrita e consulta confirmadas.

InfluxDB

Armazena séries temporais e responde a consultas por `message_id`. A implementação utilizada é compatível com a API 2.x e interrompe com mensagem clara se outra família for detectada.

CLI Oferece menu, painel de estado, testes, rastreamento e explicações breves para apresentação.

4 Contrato e qualidade dos dados

4.1 Campos

Tabela 2: Contrato interno da leitura.

Campo	Tipo	Unidade	Obrig.	Regra e armazenamento
<code>device_id</code>	texto	—	sim	Não vazio; tag
<code>message_id</code>	UUID	—	sim	UUID válido; field e chave de deduplicação local
<code>frame_counter</code>	inteiro	frame	rec.	Field; auxilia detecção de lacuna e ordem
<code>gateway_id</code>	texto	—	não	Tag
<code>site</code>	texto	—	não	Tag de baixa cardinalidade
<code>source</code>	enum	—	sim	<code>simulator</code> ou <code>lorawan-real</code> ; tag
<code>sent_at</code>	ISO 8601	UTC	sim	Exige timezone; field
<code>received_at</code>	ISO 8601	UTC	sim no fluxo	Timestamp do ponto
<code>temperature_c</code>	float	°C	não	−20 a 80 aceito; fora rejeitado; field

Campo	Tipo	Unidade	Obrig.	Regra e armazenamento
ph	float	escala pH	não	0 a 14; field
turbidity_ntu	float	NTU	não	Negativo rejeitado; field
conductivity_ us_cm	float	$\mu\text{S}/\text{cm}$	não	Negativo rejeitado; field
dissolved_ oxygen_mg_l	float	mg/L	não	Negativo rejeitado; field
rsi_dbm	float	dBm	não	Field; simulado nesta rodada
snr_db	float	dB	não	Field; simulado nesta rodada
distance_m	float	m	não	Field; simulado nesta rodada

Os limites são regras operacionais da POC e não critérios legais de qualidade da água. A validação separa formato inválido de valor ambiental suspeito. Por exemplo, **ph="abc"** é rejeitado; **ph=13.5** é numérico e aceito com alerta contextual. **NaN**, infinito, booleanos no lugar de números e turbidez negativa são rejeitados. Uma variável ambiental ausente permanece ausente e não é convertida em zero.

4.2 Tempo, identidade e repetição

Os timestamps usam ISO 8601 com timezone e são normalizados em UTC. **sent_at** representa a geração no dispositivo; **received_at**, o recebimento pelo envelope da infraestrutura. O timestamp do ponto é **received_at**. A latência entre máquinas só pode ser interpretada se seus relógios estiverem sincronizados.

O UUID da mensagem é a identidade global utilizada no rastreamento. Como QoS 1 pode repetir uma entrega, o consumer mantém deduplicação local. Uma lacuna em **frame_counter** indica possível ausência, mas não prova perda de rádio: reinício do dispositivo, rollover e retenção do broker também devem ser considerados no sistema definitivo.

5 Modelo de dados no InfluxDB

A instância detectada na execução foi InfluxDB OSS **v2.9.1**. A configuração exige URL, token, organização e bucket. O measurement é **water_quality**.

As tags são **device_id**, **gateway_id**, **site** e **source**, pois são dimensões usadas para agrupar e filtrar. Variáveis numéricas, metadados de sinal, latência, distância e contador são fields. O UUID da mensagem também é field: transformá-lo em tag criaria uma série distinta para cada mensagem e elevaria a cardinalidade.

Um ponto do InfluxDB é identificado por measurement, conjunto de tags e timestamp. Escrever novamente essa identidade pode mesclar campos. A POC reduz ambiguidades com timestamps precisos e deduplicação pelo UUID antes da escrita. A retenção deve ser escolhida segundo prazo acadêmico, volume, necessidade histórica e política institucional; ela não é fixada como requisito científico universal.

O tamanho do lote, intervalo de flush, timeout e política de retry são parâmetros operacionais. A comparação experimental da Seção 8 mostra que o lote altera fortemente a vazão, mas a seleção definitiva requer carga sustentada e recursos semelhantes aos da implantação.

6 MQTT e preparação para LoRaWAN

O contrato de comunicação inclui host, porta, tópico, QoS, autenticação, timeout e reconexão. Nenhum endereço é fixado no código: `MQTT_HOST` e `INFLUX_URL` podem apontar para outras máquinas. Credenciais ficam em `.env`, arquivo ignorado pelo Git.

No fluxo futuro, LoRa é a camada de rádio; LoRaWAN define acesso e gerenciamento da rede; a sonda é o End Device; o gateway encaminha quadros pela Internet; Network Server e Application Server processam e entregam o uplink; MQTT disponibiliza esse uplink à aplicação; e o InfluxDB preserva a série temporal. The Things Stack e ChirpStack diferem em tópicos, autenticação e envelopes. A fronteira do parser permite adaptar essas diferenças.

O hardware definitivo ainda exige dispositivo e gateway compatíveis com a região, conectividade, credenciais, DevEUI, JoinEUI/AppEUI quando aplicável, AppKey em OTAA, plano regional definido segundo a regulamentação aplicável, decoder, tópico, segurança, estratégia de retransmissão, intervalo de uplink, relógio e orçamento energético.

7 Metodologia de teste

7.1 Ambiente

Tabela 3: Ambiente efetivamente utilizado.

Elemento	Versão ou configuração
Sistema operacional	Windows
Python	3.14.6
pytest	9.1.1
Eclipse Mosquitto	2.1.2
InfluxDB OSS	v2.9.1
MQTT principal	127.0.0.1:1883, QoS 1
Broker controlado de falha	127.0.0.1:1884
Measurement	<code>water_quality</code>

7.2 Critério end-to-end

Para cada mensagem, o teste:

1. gera uma leitura com UUID único;
2. publica o envelope no broker MQTT real;

3. aguarda o subscriber do consumer;
4. executa parser, validação e controle de duplicidade/ordem;
5. grava a leitura no InfluxDB real;
6. consulta o InfluxDB pelo UUID;
7. compara campos originais e recuperados.

Se uma das 100 mensagens esperadas não for encontrada, o teste de 100 termina como falha. A aprovação não é inferida da ausência de exceções.

7.3 Métricas

A vazão é o total confirmado dividido pelo tempo do ensaio. A perda é a diferença entre geradas e localizadas no banco. Os percentis P50, P95 e P99 são calculados sobre as latências observadas. No fluxo completo, a latência por mensagem corresponde ao intervalo entre geração e escrita registrada pelo consumer; a consulta posterior fornece a prova de existência. Os horários de todos os estágios são preservados no rastreamento.

7.4 Falhas controladas

Para o InfluxDB, o teste utiliza um destino indisponível, verifica a persistência no SQLite, restaura o writer correto, reprocessa a fila, consulta o UUID e só então remove a pendência. Para MQTT, uma instância real do Mosquitto é iniciada em porta isolada, encerrada e iniciada novamente. O consumer precisa detectar a queda, permanecer vivo, reconectar, receber nova mensagem e confirmá-la no banco.

8 Resultados

8.1 Suíte automatizada

A execução final de `python -m pytest -ra`, com `RUN_INTEGRATION=1`, coletou 22 testes: **20 passaram, 2 foram ignorados** e nenhum falhou, em 9,69 segundos. Os ignorados foram o teste de hardware LoRaWAN, por ausência de dispositivo e gateway, e a carga extensa, que exige ativação explícita para não sobrecarregar a máquina. Eles não foram contabilizados como aprovados.

8.2 Fluxo completo

Não houve perda nos dois ensaios. A latência, porém, cresceu de forma pronunciada na rajada maior. Isso evidencia formação de fila no caminho completo, que nesta versão escreve e registra individualmente cada mensagem. O teste de lote, apresentado adiante, isola o potencial do banco e mostra que o InfluxDB aceita vazão muito superior quando os pontos são agrupados.

Tabela 4: Testes funcionais e de falha.

Teste	Resultado	Evidência principal
MQTT isolado	PASS	UUID idêntico; 19,884 ms
InfluxDB isolado	PASS	25,123456 °C e pH 7,654321 sem diferença
End-to-end, 100	PASS	100 geradas e 100 consultadas
Duplicidade e ordem	PASS	1 duplicata, 1 fora de ordem e 3 lacunas detectadas
Fila e recuperação do InfluxDB	PASS	pendência gravada, consultada e removida
Queda e retorno do MQTT	PASS	queda detectada, reconexão e confirmação no banco
Reinício do consumer	PASS	leituras antes/depois confirmadas; fila 0
Evento ambiental simulado	PASS	60/60 recebidas; 9 alterações sinalizadas
Hardware LoRaWAN	SKIPPED	requisito físico ausente

Tabela 5: Execuções end-to-end.

Mens.	Sondas	Confirm.	Perda	msg/s	Média (ms)	P95 (ms)	P99 (ms)
100	100	100	0	20,259	1.423,200	2.217,452	2.298,360
1.000	10	1.000	0	17,983	17.566,275	28.989,733	29.555,505

8.3 Escala por número de dispositivos

Cada dispositivo publicou uma mensagem. Assim, este ensaio representa rajadas de tamanho crescente, e não carga sustentada por longos períodos.

Tabela 6: Escala de uma mensagem por dispositivo.

Sondas	Mens.	Armaz.	Perda	msg/s	Média (ms)	P95 (ms)
1	1	1	0	4,583	17,432	17,432
10	10	10	0	16,542	172,858	244,340
50	50	50	0	19,232	777,284	1.216,231
100	100	100	0	19,803	1.319,238	2.121,202
500	500	500	0	20,149	6.844,378	11.723,254

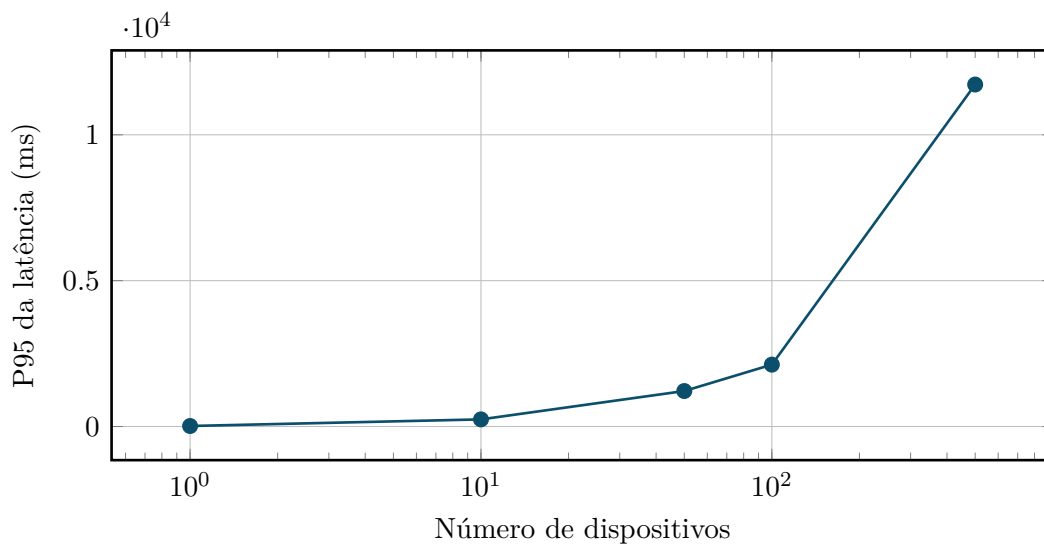


Figura 2: Crescimento do P95 nas rajadas com mais dispositivos. Dados medidos; percentil calculado.

A vazão se aproximou de 20 mensagens por segundo, enquanto o P95 aumentou até 11,723 s com 500 mensagens simultâneas. A conclusão sustentada pelos dados é que o pipeline preservou todas as mensagens dessa rajada, mas acumulou espera.

8.4 Escrita em lote no InfluxDB

O teste escreveu 1.000 pontos para cada tamanho de lote e consultou posteriormente os 1.000 identificadores. Não houve erro nem ausência.

O lote 1.000 obteve a maior vazão agregada; o lote 10 apresentou a menor duração média por chamada. A escolha operacional depende do equilíbrio entre vazão, memória, tempo de espera para formar o lote e tolerância à perda de um lote em trânsito. A execução não autoriza generalização para outro hardware.

Tabela 7: Comparação de lotes no InfluxDB.

Lote	Localizados	Erros	Tempo (s)	pontos/s	Média/chamada (ms)
1	1.000	0	2,377	420,639	2,377
10	1.000	0	0,169	5.917,583	1,689
100	1.000	0	0,062	16.124,481	6,200
500	1.000	0	0,046	21.903,694	22,825
1.000	1.000	0	0,044	22.669,311	44,109

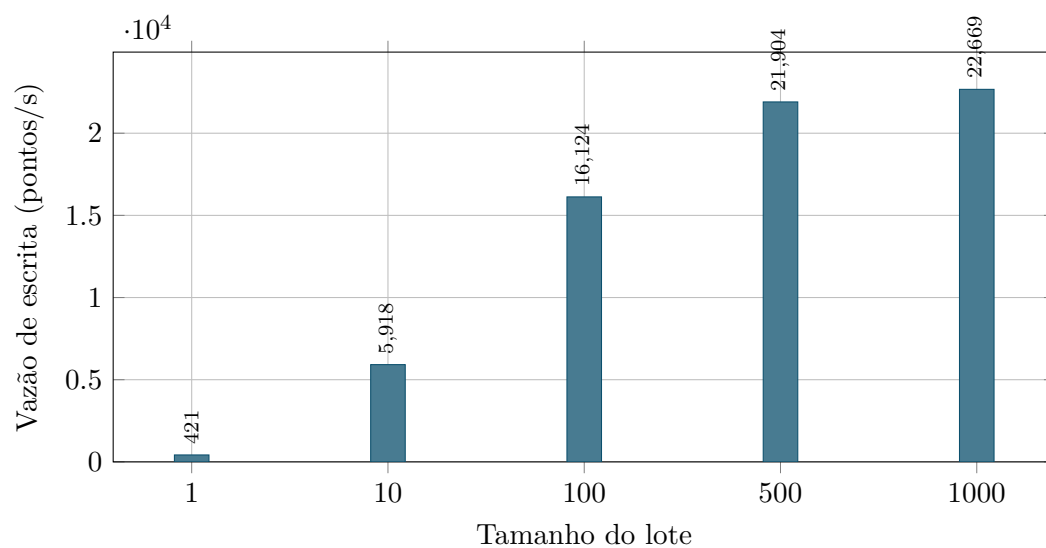


Figura 3: Vazão de escrita direta por tamanho de lote.

8.5 Frequência solicitada e observada

Foram publicadas amostras curtas de 20 mensagens em cada taxa. O limite de segurança era 1.000 mensagens por segundo e interrupção caso P95 ultrapassasse 10 s.

Tabela 8: Teste indicativo de frequência.

Solicitada	Observada	Armaz.	Perda	Média (ms)	P95 (ms)
1	0,974	20	0	17,131	18,463
10	8,274	20	0	16,537	17,372
100	15,624	20	0	329,667	464,704
500	18,686	20	0	312,171	443,791
1.000	18,575	20	0	294,704	436,723

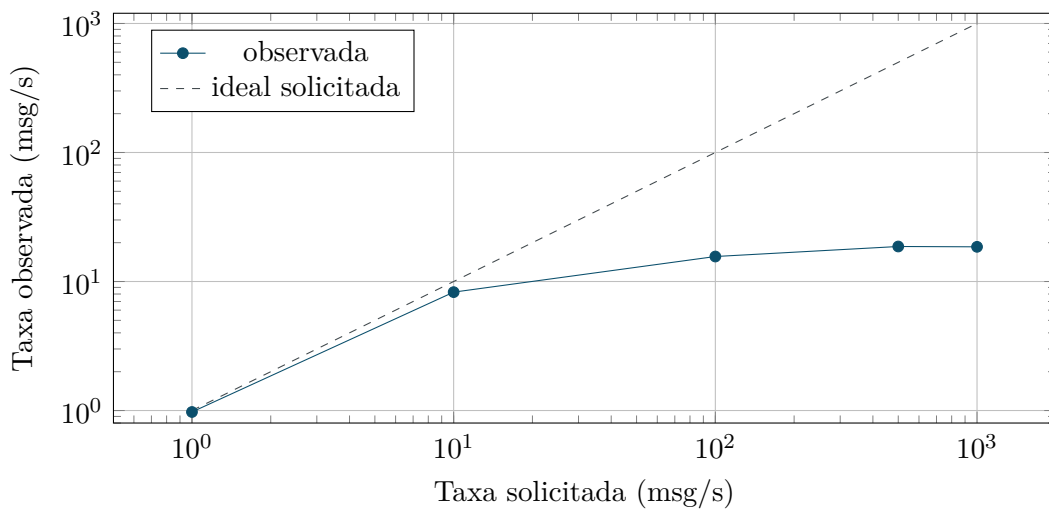


Figura 4: Taxa solicitada e vazão end-to-end observada em amostras curtas.

As amostras indicam saturação do pipeline individual perto de 19 mensagens por segundo nesse ambiente. Como cada ponto contém apenas 20 mensagens, o ensaio localiza a tendência, mas não determina uma taxa sustentável de produção. Essa determinação exige duração maior, monitoramento de CPU, memória e disco e repetição estatística.

8.6 Falha e recuperação

No teste de indisponibilidade do InfluxDB, uma mensagem válida foi enviada a um destino deliberadamente indisponível. Ela permaneceu no SQLite, foi reprocessada após a restauração, localizada por consulta e removida da fila. Resultado: **PASS**.

No teste MQTT, o broker 2.1.2 controlado na porta 1884 foi encerrado de fato. O consumer registrou a desconexão e permaneceu vivo. Depois da reinicialização, reconectou, recebeu o UUID `f9245534-b1da-4115-abec-20f4186f097f` e o confirmou no InfluxDB. Resultado: **PASS**.

No reinício do consumer, uma leitura anterior e outra posterior ao reinício foram confirmadas e a fila terminou em zero. Resultado: **PASS**.

8.7 Evento ambiental simulado

O cenário de turbidez publicou 60 mensagens; 60 foram recebidas e 9 mudanças ultrapassaram o limiar configurado. Esse resultado comprova apenas que o software detecta uma alteração temporal sintética. Não comprova poluição real, calibração de sensor ou relação causal ambiental.

9 Discussão

9.1 O que foi demonstrado

O InfluxDB recebeu e recuperou os valores sem alteração nos testes controlados. O fluxo completo preservou 100 e 1.000 mensagens, e a busca por UUID impediu que um simples retorno de escrita fosse usado como evidência suficiente. O sistema também reagiu conforme especificado a duplicidade, contador fora de ordem, lacunas e falhas temporárias.

Os dados mostram dois comportamentos distintos. No pipeline completo, a vazão ficou próxima de 18–20 mensagens por segundo e a latência cresceu com a rajada. Na escrita direta em lote, o banco recebeu dezenas de milhares de pontos por segundo. Portanto, nessa implementação, o principal espaço de otimização está no agrupamento e no caminho síncrono de processamento, e não em uma incapacidade demonstrada do InfluxDB de aceitar os volumes ensaiados.

9.2 Ameaças à validade

- Os testes foram executados em uma única máquina, compartilhando CPU, memória e disco.
- A execução de escala usou uma mensagem por dispositivo; não representa horas de tráfego contínuo.
- A comparação de frequência usou somente 20 mensagens por ponto.
- Não houve repetição suficiente para intervalos de confiança.
- Não foram monitorados CPU, memória, IOPS ou cache do sistema operacional.
- A comunicação local não inclui atraso, jitter e perda de uma LAN entre hosts.
- Os relógios não foram avaliados entre máquinas distintas.
- Não foram executadas 10 mil a 500 mil mensagens nesta rodada.
- Nenhum resultado caracteriza rádio LoRaWAN físico.

Consequentemente, os valores são evidência da POC neste ambiente, não um dimensionamento de produção.

10 Reprodução

10.1 Obtenção e preparação

Em PowerShell, clone o repositório e prepare um ambiente virtual:

```
git clone https://github.com/kyl8/m.git
```

```
cd m
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
Copy-Item .env.example .env
```

Instale o Eclipse Mosquitto diretamente pelo instalador oficial e confirme o serviço:

```
Get-Service mosquitto
Test-NetConnection localhost -Port 1883
```

Instale o InfluxDB OSS 2.x pelo ZIP oficial para Windows, execute `influxd.exe` e faça a configuração inicial com a CLI `influx setup`. A CLI e o servidor podem ser distribuídos separadamente. Crie uma organização e um bucket; o nome sugerido é `water-quality`. Nunca versione o token.

```
influx setup '
  --host http://localhost:8086 '
  --org portos-conectados '
  --bucket water-quality '
  --username SEU_USUARIO '
  --password SUA_SENHA

Invoke-RestMethod http://localhost:8086/health
```

Edite o arquivo `.env`:

```
MQTT_HOST=localhost
MQTT_PORT=1883
MQTT_TOPIC=portos/lorawan/uplink
MQTT_QOS=1

INFLUX_URL=http://localhost:8086
INFLUX_TOKEN=TOKEN_LOCAL
INFLUX_ORG=portos-conectados
INFLUX_BUCKET=water-quality
```

10.2 Sequência mínima de validação

```
scripts\check_environment.bat
scripts\test_mqtt.bat
scripts\test_influx.bat
scripts\test_end_to_end.bat
scripts\demo.bat
```

O primeiro teste prova ida e volta no broker. O segundo escreve e consulta valores conhecidos. O terceiro executa simulador, MQTT, consumer, InfluxDB, consulta

e comparação. O menu da demonstração reúne esses comandos e o rastreamento de mensagens.

Para a suíte automatizada completa:

```
$env:RUN_INTEGRATION="1"
python -m pytest -ra
```

Para carga e comparações explícitas:

```
python -m tests.load.load_test --devices 10 --messages 1000
python -m tests.load.load_test --scale --messages-per-device 1
python -m tests.load.test_batch --total 1000
python -m tests.load.test_frequency --messages 20
scripts\test_failures.bat
scripts\test_mqtt_reconnect.bat
scripts\test_consumer_restart.bat
```

Volumes maiores devem ser aumentados progressivamente. A pessoa responsável pelo ensaio deve observar fila, latência, CPU, memória e espaço em disco e interromper antes de comprometer a máquina.

10.3 Execução cotidiana

Em dois terminais:

```
scripts\start_consumer.bat

scripts\start_simulator.bat --devices 10 --messages 100 \
    --profile stress
```

O perfil **realistic** usa variação gradual e intervalo configurável; o perfil **stress** testa somente o backend e não representa tráfego no ar LoRaWAN.

10.4 Topologia entre computadores

Para um teste distribuído, configure a máquina A para executar o simulador, a B para hospedar o Mosquitto e a C para executar consumer e InfluxDB. Substitua `localhost` pelos IPs reais, restrinja firewall e autenticação e sincronize os relógios por NTP. O relatório deve registrar host e IP de cada componente, UUID, `sent_at`, recebimento pelo consumer e confirmação no banco.

Esse ensaio deve ser chamado **NETWORK DATA PIPELINE TEST**. Ele comprova comunicação IP real entre máquinas, não alcance LoRaWAN.

11 Substituição futura pelo LoRaWAN real

O simulador publica o mesmo tipo de envelope que o consumer espera receber de uma integração. Para migrar:

1. registrar o dispositivo físico no The Things Stack ou ChirpStack;
2. configurar frequência e região segundo hardware, local e regulamentação aplicável;
3. configurar decoder do payload e integração MQTT do Application Server;
4. apontar `MQTT_HOST`, tópico e credenciais para o Network Server;
5. adaptar somente `consumer/parser.py` se o envelope diferir;
6. marcar `source=lorawan-real`;
7. repetir testes de integridade antes do ensaio de campo.

No ensaio de campo, o gateway deve permanecer fixo e a sonda deve ser deslocada por pontos progressivos, definidos de acordo com condições reais e legais. Em cada ponto devem ser registrados distância, enviados, recebidos, PDR, RSSI, SNR, spreading factor, gateway e tempo. Nenhum número deve ser preenchido antes da medição.

12 Conclusão

A POC estabeleceu uma bancada terminal verificável para o trecho posterior ao Network Server. O fluxo MQTT-consumer-InfluxDB operou com serviços reais, e a persistência foi demonstrada por consultas e comparações de conteúdo. Não houve perda nos volumes executados, embora a latência tenha crescido com rajadas maiores. O teste de batch mostrou que agrupar pontos aumenta substancialmente a vazão direta de escrita, fornecendo uma direção objetiva para a próxima iteração.

As rotinas de falha preservaram dados no SQLite, reconectaram ao MQTT e confirmaram o banco antes de declarar recuperação. O contrato de dados, os limites de validação e a separação entre simulação e medição física ficaram formalizados.

Permanecem como próximos passos os ensaios longos e repetidos, volumes de 10 mil a 500 mil mensagens, monitoramento de recursos, uma topologia real em múltiplos computadores e o teste de campo LoRaWAN com rádio e gateway físicos. Até que isso ocorra, alcance, PDR e qualidade real do enlace permanecem **NÃO TESTADO**.

A Arquivos de evidência selecionados

As cópias versionadas em `results/evidence/` preservam os dados utilizados neste relatório:

- `mqtt_basic.json` e `influx_basic.json`;
- `end_to_end_100.json` e `end_to_end_1000.json`;
- `batch_comparison.json` e `frequency_comparison.json`;
- `failure_recovery.json` e `mqtt_reconnect.json`;
- `consumer_restart.json`, `message_rules.json` e `pollution_event.json`.

Os arquivos brutos completos em `evidence/` e `reports/` são gerados novamente a cada execução e, por padrão, não são versionados para evitar crescimento indefinido do repositório.

B Referências técnicas

Referências

- [1] InfluxData. *Install InfluxDB OSS v2*. Disponível em: <https://docs.influxdata.com/influxdb/v2/install/>. Acesso em: 8 out. 2026.
- [2] InfluxData. *InfluxDB schema design*. Disponível em: <https://docs.influxdata.com/influxdb/v2/write-data/best-practices/schema-design/>. Acesso em: 8 out. 2026.
- [3] Eclipse Foundation. *Eclipse Mosquitto*. Disponível em: <https://mosquitto.org/>. Acesso em: 8 out. 2026.
- [4] OASIS. *MQTT Version 5.0*. OASIS Standard, 2019. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. Acesso em: 8 out. 2026.
- [5] LoRa Alliance. *LoRaWAN L2 1.0.4 Specification*. Disponível em: <https://resources.lora-alliance.org/technical-specifications/ts001-1-0-4-lorawan-l2-1-0-4-specification>. Acesso em: 8 out. 2026.